Human-perceived Difficulty of Chess Positions: Survey and Preliminary Experiments

Zala Urh
zu0476@student.uni-lj.si
Faculty of Computer and Information Science, University of Ljubljana
Večna pot 113
SI-1000 Ljubljana, Slovenia

Matej Belšak
mb4390@student.uni-lj.si
Faculty of Computer and Information Science, University of Ljubljana
Večna pot 113
SI-1000 Ljubljana, Slovenia

Luka Bajić
luka.bajic@fri.uni-lj.si
Faculty of Computer and Information Science, University of Ljubljana
Večna pot 113
SI-1000 Ljubljana, Slovenia

Ivan Bratko
ivan.bratko@fri.uni-lj.si
Faculty of Computer and Information Science, University of Ljubljana
Večna pot 113
SI-1000 Ljubljana, Slovenia

ABSTRACT

In this paper we survey existing approaches for estimating difficulty of chess positions as perceived by human players. Additionally, we offer our own implementation of rating prediction on chess puzzles using hand-crafted features, primarily based on the properties of the search tree and on performance of weak engines. We explain both classifiers and regressors using SHAP.

KEYWORDS

chess, classification, regression, ranking, feature engineering

1 INTRODUCTION

In this paper we compare various methods for determining the difficulty of a given chess position for human players, using chess engine evaluations and other attributes of the position. This problem can be defined in the following three ways:

- classification: the model classifies positions into pre-defined classes (e.g. easy/medium/hard), or binned ELO ratings.
- regression: the model directly predicts an estimated ELO rating of the position.
- ranking: the model orders a set of positions from easiest to hardest, or ranks a single position in relation to all positions in the training set.

Full implementation is available at [1].

2 RELATED WORK

Recent approaches for estimating the difficulty of a chess position, e. g. [6] are largely based on the deep learning paradigm and the input of these models consists of some encoding of the chess position, but no information that would provide interpretability for human players. In this paper we therefore focus on finding features that achieve good classification score, but also provide a meaningful interpretation regarding the reasons why a position is considered difficult or easy for human players.

Our approaches are based on insights provided by [1], [2], [10]. These studies developed numerous attributes that can be obtained by performing a chess engine search of certain depth, starting in the initial position we wish to evaluate. The search is constrained in such a way that only moves which human players would typically assess as meaningful are explored, which aims to provide an estimate of how complex the search tree around the position is.

3 METHODOLOGY

3.1 Data

To train and evaluate our models, we use a subset of the Lichess puzzle dataset [2] containing 3000 puzzles with ratings ranging between 399 and 3092. We discard all columns except FEN, rating, and themes. Furthermore, we only keep a subset of themes which have a precise definition (for a full list, see the implementation). For classification we create classes "easy", "medium" and "hard" by splitting this interval equidistantly. The dataset is split into training set and test set with the ratio 80:20. The data is standardized, meaning we transform it in such a way that it follows a standard normal distribution with zero mean and unit variance. This is required by distance-based models to ensure that each attribute is treated as equally important.

3.2 Evaluation Metrics

For classification problem we use the standard metrics: classification accuracy, F1 score and AUC. For regression we use mean absolute error (MAE) because unlike mean squared error, it has a more intuitive interpretation on this dataset. We also report which percentage of the rating interval this MAE covers. For ranking we use Normalized Discounted Cumulative Gain (NDCG) and Kendall τ to determine the accuracy of the model's ranking against the

[1] https://github.com/bajicluka01/chess-puzzle-evaluation/tree/main

[2] https://database.lichess.org/#puzzles

ground truth, which is simply a ranking assigned by ordering the ratings in the dataset.

3.3 Atrributes

In this section we discuss the features that were used for model training in Chapter 4.

3.3.1 Meaningful Search Tree. We use the features from [10] (except *SeeminglyGood*, which is not rigorously defined and cannot be easily reproduced) with the following modification: we generate the *MeaningfulMoves* feature using Otter [9], a recently developed engine that models human play, and use these moves to compute the remaining features from [10].

This change is motivated by the following: meaningful moves in [10] are intended to represent the moves a human player would generally consider in a given position (including moves that are quickly rejected, e.g. captures that obviously do not work). However, these moves are computed directly from the engine evaluation, which renders the feature useless in many cases, e.g. in tactical positions in which there is objectively only one good move, whereas a human player clearly needs to consider multiple alternatives. Conversely, Otter [9] is a transformer-based approach that for a given position P returns a probability distribution across all legal moves in P, indicating how likely a human player is to consider each alternative. In other words, we obtain probabilities p_i with the following properties:

$$\forall i = 1, \ldots, k: \quad p_i \geq 0, \quad \sum_{L(P)} p_i = 1 \tag{1}$$

where $L(P)$ is the set of legal moves in position P and i enumerates all the moves in $L(P)$, so $|L(P)| = k$.

In our experiments we select meaningful moves by using an arbitrarily chosen cutoff $p_i > 0.1$, which based on Equation (1) gives at most 10 (but typically less) moves most likely to be considered by a human player in a given position. In our view this coincides with the number of moves an expert player typically considers.

3.3.2 Weak Engines. The following reasoning should make intuitive sense: if a weak engine (or player) can solve a given puzzle, it is more likely to be a low-rated puzzle. Therefore we can employ several weak engines and use indicators of their puzzle-solving success as a subset of features. What exactly is meant by "weak" is not of great importance (see discussion in Chapter 4.1) - in fact it is desirable to use multiple engines with varying estimated strengths [3] to obtain less correlated features.

3.3.3 Miscellaneous. In addition to the above mentioned features, we also use Stockfish's outputs, namely centipawn evaluations, win/draw/loss rate and selective depth.

Furthermore, we compute the number of pins and forks in the given position based on the following definitions.

3.4 Explainability

[4]

[3]Note that using, e.g. Stockfish with different playing strength parameters, gives unsatisfactory results. Evidently, making a strong engine play at less-than-full strength, is a harder (and perhaps less interesting) problem than implementing a weak engine from scratch.

4 RESULTS

4.1 Correlation between puzzle rating and engine Elo

The quality of the Elo rating system in chess has been called into question numerous times, e.g. in [3, 8], however it remains a generally accepted measure of playing strength. In many settings, including Lichess (hence also in the dataset used in this paper), the Glicko-2 [?] rating system is used instead [4]. On the other hand, estimation of chess engines' playing strength is generally still done using Elo [5]. Therefore it is worth investigating the correlation between an engine's rating and its ability to solve puzzles of a similar rating.

4.2 Classification

Table 1 displays the performance of the standard machine learning models [7] on a dataset with 1000 samples and all available attributes. Class distribution is 39.1% "easy", 38.9% "medium" and 22.0% "hard".

XGBoost uses 100 estimators and learning rate 0.1. SVM uses regularization parameter $C = 1$ and balanced class weight. Random Forest uses 100 estimators. kNN uses 10 nearest neighbors and Euclidean distance. MLP uses $layers = [2048, 1024, 512, 256, 128]$, ReLU activation function, ADAM solver, regularization $\alpha = 0.001$ and batch size 512. Default values are used for all remaining parameters. Ensemble uses a soft voting scheme on the outputs of all other models, meaning that probabilistic predictions are aggregated and finally converted into a class value. Misclassifications are discussed in Appendix 2.

Table 1: Classification scores on 1k dataset.

model	CA	F1	AUC
XGBoost	0.575	0.574	0.756
kNN	**0.635**	**0.621**	0.773
Random Forest	0.620	0.616	0.772
Gradient Boosting	0.575	0.563	0.758
MLP	0.630	**0.621**	0.770
SGD	0.550	0.551	0.693
SVM	0.595	0.574	0.756
Ensemble	0.615	0.612	**0.779**

In Table 2 we show the results of an ablation study aimed to determine the utility of each subset of attributes. We use the best performing model in terms of CA from the previous experiment: kNN with $k = 10$. Baseline refers to simple attributes, such as centipawn evaluations and number of searched nodes for each depth. These attributes are used in each experiment. Study refers to attributes from [10].

[4]Whether applying any rating system to an individual position rather than a player/engine even makes sense is a separate discussion, which we do not engage in here.

[5]Furthermore, authors often estimate their engine's rating in non-standardized ways, which makes ratings across different engines difficult to compare

Table 2: Ablation study on 1k dataset.

model	CA	F1	AUC
baseline	0.565	0.561	0.671
counts	0.555	0.545	0.688
study	0.565	0.548	0.716
Stockfish	0.565	0.557	0.752
themes	0.565	0.554	0.704
counts, study	0.530	0.508	0.723
counts, Stockfish	0.575	0.563	0.716
counts, themes	0.565	0.553	0.713
study, Stockfish	0.580	0.561	0.766
study, themes	0.58	0.570	0.717
all attributes	0.635	0.621	0.773

This confirms the usefulness of attributes from [10] and also confirms the usefulness of themes. The contribution of solving success is negligible, possibly due to an inherent flaw in the implementation of Stockfish playing strength: weaker play is simulated by limiting the depth of the search tree and imposing a time constraint on the search, however, this might not accurately simulate the puzzle-solving ability of a player with a certain ELO, rendering the attributes useless.

4.3 Regression

Regression results are shown in Table 3. XGBoost uses 100 estimators and learning rate 0.1. MLP uses $layers = [256, 256, 128, 128, 128, 64]$, ReLU activation function, adaptive learning rate, ADAM solver, regularization parameter $\alpha = 0.001$ and batch size 256. Default values are used for all remaining parameters. Ensemble model performs unweighted averaging on the outputs of all other models.

Table 3: Regression scores on 1k dataset.

model	MAE	% of interval
XGBoost	329.836	12.2
Linear regression	321.600	11.9
MLP	317.419	11.8
Bayes Ridge	314.587	11.7
Ensemble	**310.385**	**11.5**

5 DISCUSSION

5.1 Misclasifications

6 CONCLUSIONS

Future work would consist of checking if these approaches are suitable also for strategic/nondescript positions, rather than just puzzles. Here, the main challenge is in assigning the ground truth ELO to these positions, since consensus among human players is more difficult to achieve.

Additionally, Maia chess engine [5] [11] could be used as an alternative to Stockfish, as it is designed to approximate human play more closely. However, we ran a few experiments on known positions, and again this similarity to human-like play does not directly translate to human-like puzzle solving, so additional model training may be required, which is outside the scope of this paper.

REFERENCES

[1] Ivan Bratko, Dayana Hristova, and Matej Guid. 2016. *Search Versus Knowledge in Human Problem Solving: A Case Study in Chess.* Vol. 27. 569–583. doi:10.1007/978-3-319-38983-7_32

[2] Ivan Bratko, Dayana Hristova, and Matej Guid. 2021. Predicting Problem Difficulty in Chess. In *Human-Like Machine Intelligence*, Stephen Muggleton and Nicholas Chater (Eds.). Oxford University Press. arXiv:https://academic.oup.com/book/0/chapter/350718010/chapter-pdf/43431216/oso-9780198862536-chapter-24.pdf doi:10.1093/oso/9780198862536.003.0024

[3] Mark E Glickman and Albyn C Jones. 1999. Rating the chess rating system. *Chance-Berlin Then New York*- 12 (1999), 21–28.

[4] Scott Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. arXiv:1705.07874 [cs.AI] https://arxiv.org/abs/1705.07874

[5] Reid McIlroy-Young, Siddhartha Sen, Jon Kleinberg, and Ashton Anderson. 2020. Aligning Superhuman AI with Human Behavior: Chess as a Model System. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '20)*. ACM, 1677–1687. doi:10.1145/3394486.3403219

[6] Szymon Miłosz and Paweł Kapusta. 2024. Predicting Chess Puzzle Difficulty with Transformers. arXiv:2410.11078 [cs.LG] https://arxiv.org/abs/2410.11078

[7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[8] Danilo Machado PIRES and Júlio Sílvio de Sousa BUENO FILHO. 2020. Can ELO Ratings Be Improved? A Case Study with Elite Chess Players. *Brazilian Journal of Biometrics* 38, 4 (2020), 483–505.

[9] Tarun Kumar S. 2026. Otter: A Time-Aware, History-Conditioned Human Chess AI. arXiv:2608.05206 [cs.AI] https://arxiv.org/abs/2608.05206

[10] Simon Stoiljkovikj, Ivan Bratko, and Matej Guid. 2015. A Computational Model for Estimating the Difficulty of Chess Problems. https://api.semanticscholar.org/CorpusID:204857616

[11] Zhenwei Tang, Difan Jiao, Reid McIlroy-Young, Jon Kleinberg, Siddhartha Sen, and Ashton Anderson. 2024. Maia-2: A Unified Model for Human-AI Alignment in Chess. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems.* https://openreview.net/forum?id=XWlkhRn14K